TÉCNICAS PARA ANÁLISE DE ALGORITMOS

João Pedro Oliveira Batisteli

TÉCNICAS PARA ANÁLISE DE ALGORITMOS

- Análise e determinação da função de complexidade de algoritmos ***iterativos***.

- Análise e determinação da função de complexidade de algoritmos ***recursivos***.

TÉCNICAS PARA ANÁLISE DE ALGORITMOS

- Determinar o tempo de execução de um programa pode ser um problema matemático complexo.
- A análise utiliza técnicas de matemática discreta, envolvendo contagem ou enumeração dos elementos de um conjunto:
  - manipulação de somas
  - Produtos
  - Permutações
  - Fatoriais
  - coeficientes binomiais
  - solução de equações de recorrência.

TÉCNICAS PARA ANÁLISE DE ALGORITMOS

- Não há técnica geral de análise de algoritmos: estudo caso-a-caso.

- Uso de princípios básicos de Aho, Hopcroft e Ullman(1983).
  - Utilizam as propriedades sobre a notação *O*.

TÉCNICAS PARA ANÁLISE DE ALGORITMOS

- Operações simples:
  - Leitura, escrita ou atribuição;
  - Operações aritméticas;
  - Operações lógicas;
- Podem ser consideradas como $O(1)$.
- Existem exceções para as linguagem que permitem chamadas de funções em comandos de atribuição ou quando atribuições envolvem estruturas de tamanho arbitrário.

TÉCNICAS PARA ANÁLISE DE ALGORITMOS

1) Operações simples:

- Leitura, escrita ou atribuição;
- Operações aritméticas;
- Operações lógicas;

- Podem ser consideradas como $O(1)$.
- Existem exceções para as linguagem que permitem chamadas de funções em comandos de atribuição ou quando atribuições envolvem estruturas de tamanho arbitrário.

TÉCNICAS PARA ANÁLISE DE ALGORITMOS

2) Custo de uma sequência:

- soma dos custos de cada operação da sequência, ou
- determinado pelo maior tempo de execução de qualquer comando da sequência.

3) Custo em laços de repetição:

- custo de execução dos comandos internos, multiplicado pelo número de repetições.

⚠ atenção no cálculo do número de repetições!! ⚠

ALGORITMOS ITERATIVOS

ALGORITMOS ITERATIVOS

- Analisar cada método de forma isolada, iniciando pelos métodos que não realizam chamadas a outros métodos, até alcançar o programa principal.
- Em estruturas iterativas, a análise deve partir do laço mais interno em direção aos laços externos.
- Identificar e considerar a operação mais relevante, isto é, aquela que domina o custo computacional do algoritmo.

```
int alg1(int n){
   int res = 1;
   for(int i=n; i>1; i--){
        res = res*i;
   }
   return res;
}
```

```
1. int alg1(int n){
2.    int res = 1;
3.    for(int i=n; i>1; i--){
4.         res = res*i;
5.    }
6.    return res;
7. }
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. int alg1(int n){
2.    int res = 1;
3.    for(int i=n; i>1; i--){
4.         res = res*i;
5.    }
6.    return res;
7. }
```

Atribuição e multiplicação

```
int alg1(int n){
   int res = 1;
   for(int i=n; i>1; i--){
        res = res*i;(2)
   }
   return res;
}
```

```
int alg1(int n){
   int res = 1;
   for(int i=n; i>1; i--){
      res = res*i;(2)
   }
   return res;
}
```

```
int alg1(int n){
   int res = 1;
   for(int i=n; i>1; i--){(n-1)
        res = res*i;(2)
   }
   return res;
}
```

```
int alg1(int n){
    int res = 1;
    for(int i=n; i>1; i--){
        res = res*i;
    }
    return res;
}
```

(2n-2)

```
int alg1(int n){
   int res = 1;
   for(int i=n; i>1; i--){
        res = res*i;
   }
   return res;
}
```

(2n-2)

$f(n) = (2n - 2)$, para n>1

```
1. public int alg2(int[] arr, int x){
2.   for(int i=0; i<arr.length; i++){
3.       if(arr[i] == x){
4.           return i;
5.       }
6.   }
7.   return -1;
8. }
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. public int alg2(int[] arr, int x){
2.   for(int i=0; i<arr.length; i++){
3.       if(arr[i] == x){
4.           return i;
5.       }
6.   }
7.   return -1;
8. }
```

```
1. public int alg2(int[] arr, int x){
2.   for(int i=0; i<arr.length; i++){
3.       if(arr[i] == x){ (1)
4.           return i;
5.       }
6.   }
7.   return -1;
8. }
```

```
1. public int alg2(int[] arr, int x){
2.   for(int i=0; i<arr.length; i++){
3.       if(arr[i] == x){ (1)
4.           return i;
5.       }
6.   }
7.   return -1;
8. }
```

```
1. public int alg2(int[] arr, int x){
2.    for(int i=0; i<arr.length; i++){ Variações
3.         if(arr[i] == x){(1)
4.              return i;(1)
5.         }
6.    }
7.    return -1;(1)
8. }
```

Vamos considerer os retornos com $O(1)$

```
1. public int alg2(int[] arr, int x){
2.    for(int i=0; i<arr.length; i++){
3.         if(arr[i] == x){ (1)
4.              return i; (1)
5.         }
6.    }
7.    return -1;(1)
8. }
```

Variações (vamos considerar tamanho n)

Pior caso: loop executa n vezes + linha 7 → f(n) = (n)*1 = n+1

```
1. public int alg2(int[] arr, int x){
2.    for(int i=0; i<arr.length; i++){
3.         if(arr[i] == x){ (1)
4.              return i; (1)
5.         }
6.    }
7.    return -1;(1)
8. }
```

Variações (vamos considerar tamanho n)

Pior caso: loop executa n vezes + linha 7 → f(n) = (n)*1 = n+1

Melhor caso: loop executa 1 vez + linha 4 → f(n) = 1*1 +1 = 2

```
1. public int alg2(int[] arr, int x){
2.    for(int i=0; i<arr.length; i++){
3.         if(arr[i] == x){ (1)
4.              return i; (1)
5.         }
6.    }
7.    return -1; (1)
8. }
```

Variações (vamos considerar tamanho n)

Pior caso: loop executa n vezes + linha 7 → f(n) = (n)*1 = n+1
Melhor caso: loop executa 1 vez + linha 4 → f(n) = 1*1 +1 = 2
Caso médio = (n+1+2)/2 = (n+3)/2

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.        for(int j = 0; j<array.length; j++){
6.          if (array[i] < array[j])
7.                quant++;
8.        }
9.        if(quant>max)
10.           max = quant;
11.       quant=0;
12.    }
13.}
```

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.           if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.           if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.         (1) if (array[i] < array[j])
7.                 quant++;
8.         }
9.     (1) if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.         (1) if (array[i] < array[j])
7.                 quant++;
8.         }
9.     (1) if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.      (n)for(int j = 0; j<array.length; j++){
6.        (1)if (array[i] < array[j])
7.                quant++;
8.        }
9.      (1)if(quant>max)
10.           max = quant;
11.       quant=0;
12.    }
13.}
```

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.     (1) if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

(n)

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.           if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

(n+1)

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

(n+1)

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

(n)

(n+1)

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

$n^2 + n$

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.         (1) if (array[i] < array[j])
7.                 quant++;
8.         }
9.     (1) if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.         (1) if (array[i] < array[j])
7.                 (2) quant++;
8.         }
9.     (1) if(quant>max)
10.         (1) max = quant;
11.         quant=0;
12.     }
13. }
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){(2)
5.         for(int j = 0; j<array.length; j++){(n)
6.             if (array[i] < array[j])
7.                 quant++;                    (3)
8.         }
9.         if(quant>max)
10.            max = quant;                    (2)
11.        quant=0;
12.    }
13.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){(2)
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

$3n^2 + 2n$

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){(2)
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

$3n^2 + 2n$

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){(2)
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.            max = quant;
11.        quant=0;
12.    }
13.}
```

$3n^2 + 2n$

Existe uma entrada onde as linhas 7 e 10 serão executadas em todas as iterações do for?

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

Será falso quando i == j

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

Será falso quando i == j

Seria executado no máximo (n-1) vezes

```
1. void alg3(int []array){
2.     int quant=0;
3.     int max=-1;
4.     for(int i = 0; i<array.length; i++){
5.         for(int j = 0; j<array.length; j++){
6.             if (array[i] < array[j])
7.                 quant++;
8.         }
9.         if(quant>max)
10.             max = quant;
11.         quant=0;
12.     }
13.}
```

Será falso quando i == j

Seria executado no máximo (n-1) vezes

Dependerá da entrada

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.          if (array[j] < array[menor])
6.                menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.          if (array[j] < array[menor])
6.                menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.          if (array[j] < array[menor])
6.                menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.          if (array[j] < array[menor])
6.                menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

0 – Não entre em pânico.
1 – O que o algoritmo faz?
2 – Marcar operações relevantes.
3 – Avaliar sequências e laços.
4 – Há variação de casos?

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.          if (array[j] < array[menor])
6.                menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.        (1)if (array[j] < array[menor])
6.                 menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.       (1) if (array[j] < array[menor])
6.                 menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

Repete-se quantas vezes?

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){ (n-1)
3.         int menor = pos;
4.         for(int j = pos+1; j<array.length; j++){
5.         (1) if (array[j] < array[menor])
6.                 menor = j;
7.         }
8.         int aux = array[menor];
9.         array[menor] = array[pos];
10.        array[pos] = aux;
11.    }
12.}
```

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){ (n-1)
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){n-1, n-1, n-3, …, 0
5.       (1) if (array[j] < array[menor])
6.               menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){ (n-1)
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){n-1, n-1, n-3, …, 0
5.       (1) if (array[j] < array[menor])
6.               menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

Temos um somatório:

$$\sum_{i=0}^{n-1} i$$

ALGUNS SOMATÓRIOS ÚTEIS

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=0}^{k} a^i = \frac{a^{k+1}-1}{a-1} (a \neq 1)$$

$$\sum_{i=0}^{k} 2^k = 2^{k+1} - 1$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=0}^{k} \frac{1}{2^i} = 2 - \frac{1}{2^k}$$

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){ (n-1)
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){n-1, n-1, n-3, ..., 0
5.       (1) if (array[j] < array[menor])
6.               menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

Temos um somatório:

$$\sum_{i=0}^{n-1} i$$

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){ (n-1)
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){n-1, n-1, n-3, ..., 0
5.       (1) if (array[j] < array[menor])
6.               menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

Temos um somatório:

$$\sum_{i=0}^{n-1} i = \frac{(n-1)*(n-1+1)}{2}$$

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){ (n-1)
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){n-1, n-1, n-3, ..., 0
5.       (1) if (array[j] < array[menor])
6.               menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

Temos um somatório:

$$\sum_{i=0}^{n-1} i = \frac{(n-1)*(n)}{2}$$

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){ (n-1)
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){n-1, n-1, n-3, ..., 0
5.       (1) if (array[j] < array[menor])
6.               menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

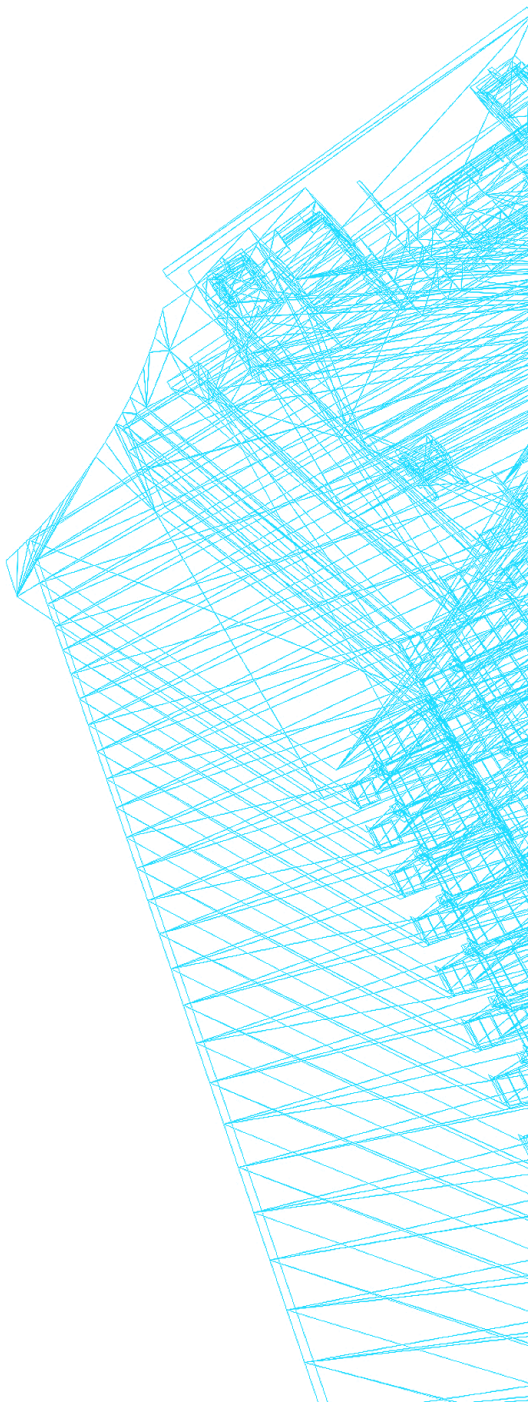
Temos um somatório:

$$\sum_{i=0}^{n-1} i = \frac{(n^2 - n)}{2}$$

```
1. void alg4(int []array){
2.     for(int pos = 0; pos<array.length-1; pos++){
3.       int menor = pos;
4.       for(int j = pos+1; j<array.length; j++){
5.          if (array[j] < array[menor])
6.                menor = j;
7.       }
8.       int aux = array[menor];
9.       array[menor] = array[pos];
10.      array[pos] = aux;
11.    }
12.}
```

E se considerarmos também as trocas de posição?

ALGORITMOS RECURSIVOS



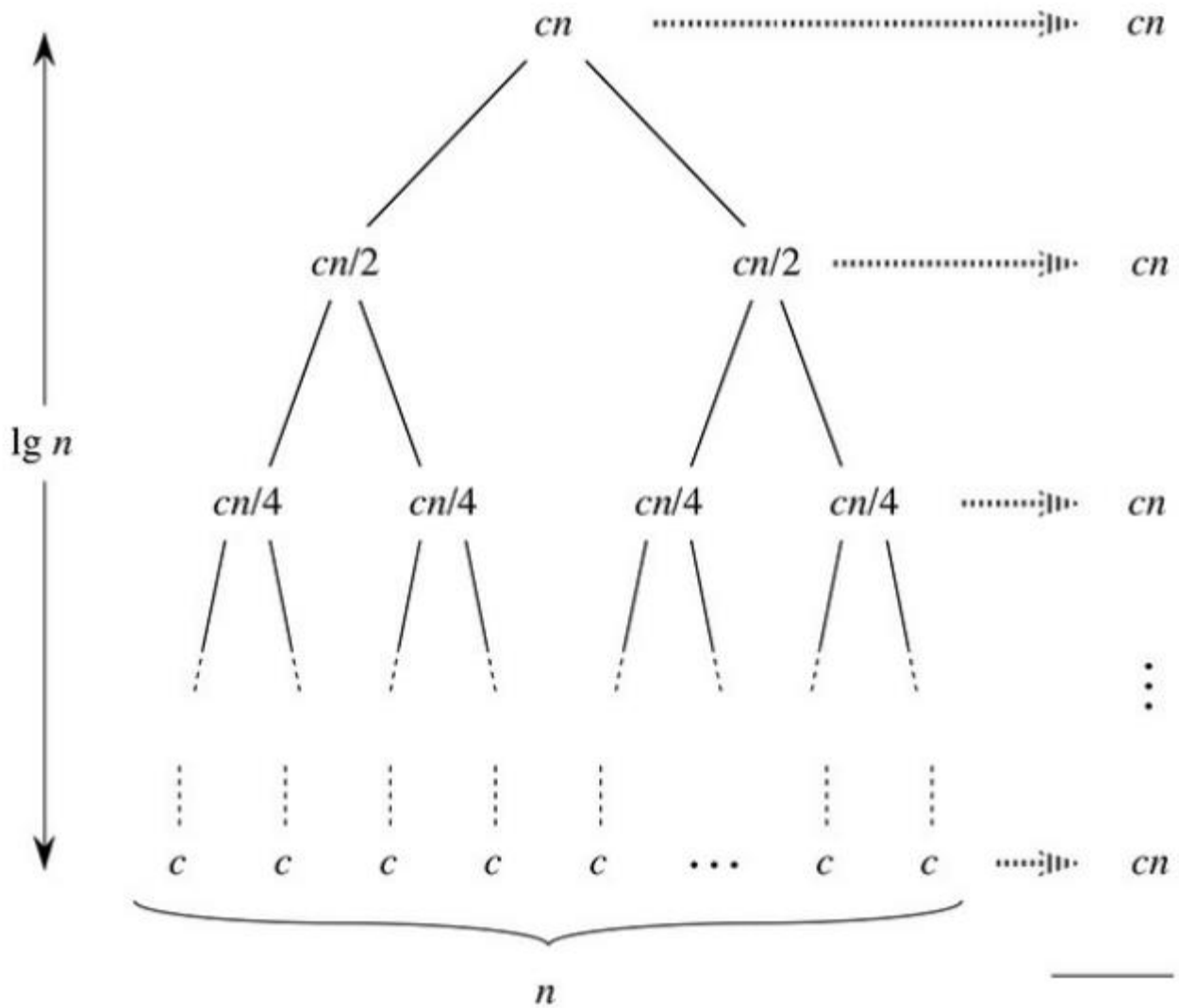
TÉCNICAS PARA ANÁLISE DE ALGORITMOS

- Quantas vezes uma chamada recursiva executará?
  - Representação pela função matemática do método.

- Teremos uma ***equação de recorrência*** contendo ***base*** e ***recursividade.***
  - *Resolver a recorrência.*

TÉCNICAS PARA ANÁLISE DE ALGORITMOS

De onde surge o $log_2(n)$ do quicksort, mergesort e heapsort?


cn
cn
cn/2
cn/2
cn
lg n
cn/4
cn/4
cn/4
cn/4
cn
c c c c c ... c c
cn
n

```
1. int rec1(int n){
2.   if(n==0)
3.       return 1;
4.   else
5.       return n * rec1(n-1);
6. }
```

~~0 – Não entre em pânico~~

1 – Base e recursividade

Base:

2 – Definir ‘n’ e custo

Recursividade:

3 – Operações relevantes?

4 – Marcar operações

5 – Avaliar sequências

6 – Custo da recursividade

```
int rec1(int n){
   if(n==0)  ← S(0)
        return 1;
   else
        return n * rec1(n-1);
}
```

```
int rec1(int n){
    if(n==0)
        return 1;
    else
        return n * rec1(n-1);
}
```

$S(0) = 1$

```
int rec1(int n){
    if(n==0)
        return 1;  ← S(0) = 1
    else  ← S(n)
        return n * rec1(n-1);
}
```

```
int rec1(int n){
    if(n==0)
        return 1;
    else
        return n * rec1(n-1);
}
```

S(0) = 1

S(n)

1 operação

```
int rec1(int n){
    if(n==0)
        return 1;
    else
        return n * rec1(n-1);
}
```

$S(0) = 1$

$S(n) = 1 + S(n+1)$

EQUAÇÃO DEFINIDA POR PARTES

$$s(n) = \begin{cases} 1, & n = 0 \\ 1 + s(n-1), & n > 0 \end{cases}$$

$$T(n) = \begin{cases} 1, & n = 0 \\ 1 + T(n-1), & n > 0 \end{cases}$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

EQUAÇÃO DEFINIDA POR PARTES

$$s(n) = \begin{cases} 1, & n = 0 \\ 1 + s(n-1), & n > 0 \end{cases}$$

$$T(n) = \begin{cases} 1, & n = 0 \\ 1 + T(n-1), & n > 0 \end{cases}$$

É uma boa maneira de anotar o tempo de execução de algoritmos de divisão e conquista

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$S(n-1) = S(n-2) + 1$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 1 + 1$$

$$S(n-1) = S(n-2) + 1$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \\ \qquad\quad = S(n-2) + 2 \end{cases}$$

$$S(n-1) = S(n-2) + 1$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2 \qquad S(n-2) = S(n-3) + 1$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2 \qquad S(n-2) = S(n-3) + 1$$

$$= S(n-3) + 1 + 2$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2 \qquad S(n-2) = S(n-3) + 1$$

$$= S(n-3) + 3$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2$$

$$= S(n-3) + 3$$

Conseguem reconhecer o padrão?

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2$$

$$= S(n-3) + 3$$

Conseguem reconhecer o padrão?

$$F.G = S(n-i) + i$$

F.G = Fórmula Geral
Descreve o padrão observado antes de aplicar a condição inicial/final

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2$$

$$= S(n-3) + 3$$

Conseguem reconhecer o padrão?

$$F.G = S(n-i) + i$$

Até onde esse valor vai?

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2$$

$$= S(n-3) + 3$$

$$F.G = S(n-i) + i$$

Na base, $S(n-i) = S(0) \rightarrow n - i = 0 \rightarrow n = i$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2$$

$$= S(n-3) + 3$$

$$F.G = S(n-i) + i$$

Na base, $S(n-i) = S(0) \rightarrow n - i = 0 \rightarrow n = i$

Substituindo $i = n$ na F.G:

$$S(n) = S(0) + n$$

$$\begin{cases} S(0) = 1, \\ S(n) = S(n-1) + 1 \end{cases}$$

$$= S(n-2) + 2$$

$$= S(n-3) + 3$$

$$F.G = S(n-i) + i$$

Na base, $S(n-i) = S(0) \rightarrow n - i = 0 \rightarrow n = i$

Substituindo $n = i$ na F.G:

$$S(n) = S(0) + n = 1 + n$$

```
1. int rec2(int[] vet, int i){
2.   if(i==0)
3.     return vet[0];
4.   else{
5.     for(int j=0; j<vet.length; j++){
6.       vet[j] += vet[i];
7.     }
8.     return rec2(vet, i-1);
9.   }
10.}
```

~~0 – Não entre em pânico~~

1 – Base e recursividade

Base:

2 – Definir ‘n’ e custo

Recursividade:

3 – Operações relevantes?

4 – Marcar operações

5 – Avaliar sequências

6 – Custo da recursividade

```
1. int rec2(int[] vet, int i){
2.   if(i==0)
3.     return vet[0];
4.   else{
5.     for(int j=0; j<vet.length; j++){
6.       vet[j] += vet[i];
7.     }
8.     return rec2(vet, i-1);
9.   }
10.}
```

```
int rec2(int[] vet, int i){
  if(i==0)
    return vet[0];
  else{
    for(int j=0; j<vet.length; j++){
      vet[j] += vet[i];
    }
    return rec2(vet, i-1);
  }
}
```

$S(0) = 1$

```
int rec2(int[] vet, int i){
  if(i==0)
    return vet[0];
  else{
    for(int j=0; j<vet.length; j++){
      vet[j] += vet[i];
    }
    return rec2(vet, i-1);
  }
}
```

$S(0) = 1$

n

```
int rec2(int[] vet, int i){
  if(i==0)
    return vet[0];
  else{
    for(int j=0; j<vet.length; j++){
      vet[j] += vet[i];
    }
    return rec2(vet, i-1);
  }
}
```

$S(0) = 1$

$S(n) = n + S(n-1)$

```
int rec2(int[] vet, int i){
  if(i==0)
    return vet[0];
  else{
    for(int j=0; j<vet.length; j++){
      vet[j] += vet[i];
    }
    return rec2(vet, i-1);
  }
}
```

$S(0) = 1$

$S(i) = n + S(i-1)$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-1) = S(i-2) + n$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-1) = S(i-2) + n$$

$$S(i-2) + n + n$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-2) = S(i-3) + n$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-3) + n + 2n$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-3) + 3n$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-3) + 3n$$

Conseguem reconhecer o padrão?

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-3) + 3n$$

Conseguem reconhecer o padrão?

$$F.G = S(i-k) + kn$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-3) + 3n$$

$$F.G = S(i-k) + kn$$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-3) + 3n$$

$$F.G = S(i-k) + kn$$

Na base, $S(i-k) = S(0) \rightarrow i - k = 0 \rightarrow i = k$

$$\begin{cases} S(0) = 1, \\ S(i) = S(i-1) + n \end{cases}$$

$$S(i-2) + 2n$$

$$S(i-3) + 3n$$

$$F.G = S(i-k) + kn$$

Na base, $S(i-k) = S(0) \rightarrow i - k = 0 \rightarrow i = k$

Substituindo $i = k$ na F.G:

$$S(i-\mathrm{k}) = S(0) + in = 1 + in$$

EXERCÍCIO 1

```
1. int rec3(int[] vet, int a, int b){
2.     if(a==b)
3.         return vet[a];
4.     else{
5.         int c = (a+b)/2;
6.         int d = rec3(vet, a, c);
7.         int e = rec3(vet, c, b);
8.         return d+e;
9.     }
10.}
```

RETOMADA: IDEIA INTUITIVA USADA ATÉ AGORA

Como analisamos algoritmos recursivos até aqui

- Partimos do **código recursivo**
- Observamos:
  - o **caso base**
  - o **trabalho realizado em cada chamada**
- Expandimos manualmente as chamadas recursivas
- Identificamos um **padrão**
- Obtemos a **Fórmula Geral (F.G)**

RETOMADA: IDEIA INTUITIVA USADA ATÉ AGORA

Esse método funciona, mas **fica trabalhoso** para recorrências mais complexas.

MOTIVAÇÃO PARA EQUAÇÕES DE RECORRÊNCIA

Por que formalizar usando recorrências?

- Facilita a análise de:
  - divisões do problema (ex.: $n/2, n/3$)
  - algoritmos do tipo *divide and conquer*
- Evita expansões longas e propensas a erro
- Permite uso de:
  - substituição
  - árvores de recorrência
  - Teorema Mestre
- Para isso, descrevemos o algoritmo como uma **equação de recorrência**.

O QUE É UMA EQUAÇÃO DE RECORRÊNCIA

Definição

- Uma equação de recorrência descreve o custo de um algoritmo em função do custo de instâncias menores do mesmo problema.

Forma geral:

$$T(n) = \begin{cases} \textit{custo base}, & n = \textit{caso base} \\ \textit{custo recursivo} + \textit{trabalho local}, & n \neq \text{caso base} \end{cases}$$

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine
      else {
(3)     para cada um dos n elementos 'inspecione elemento';
(4)     pesquisa(n/3);
      }
    }
```

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine
      else {
(3)     para cada um dos n elementos 'inspecione elemento';
(4)     pesquisa(n/3);
      }
    }
```

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine(1)
      else {
(3)     para cada um dos n elementos 'inspecione elemento';
(4)     pesquisa(n/3);
      }
    }
```

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine
      else {
(3)     para cada um dos n elementos 'inspecione elemento';
(4)     pesquisa(n/3);
      }
    }
```

$T(1) = 1$

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine
      else {
(3)     para cada um dos n elementos 'inspecione elemento';
(4)     pesquisa(n/3);
      }
    }
```

$T(1) = 1$

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine T(1) = 1
      else {
(3)     para cada um dos n elementos 'inspecione elemento'; (n)
(4)     pesquisa(n/3); Chamada recusiva
      }
    }
```

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine
      else {
(3)     para cada um dos n elementos 'inspecione elemento';
(4)     pesquisa(n/3);
      }
    }
```

$T(1) = 1$

$T(n) = T(n/3) + n$

```
    void pesquisa(n) {
(1)   if (n <= 1)
(2)     'inspecione elemento' e termine T(1) = 1
      else {
(3)     para cada um dos n elementos 'inspecione elemento';
(4)     pesquisa(n/3); T(n) = T(n/3) + n
      }
    }
```

$$T(n) = \begin{cases} 1, & n = 1 \\ T\left(\frac{n}{3}\right) + n, & n > 0 \end{cases}$$

PASSO A: EXPANSÃO DA RECORRÊNCIA

$$T(n) = \begin{cases} 1, & n = 1 \\ T\left(\frac{n}{3}\right) + n, & n > 0 \end{cases}$$

PASSO A: EXPANSÃO DA RECORRÊNCIA

$$T(n) = T\left(\frac{n}{3}\right) + n$$

PASSO A: EXPANSÃO DA RECORRÊNCIA

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

PASSO A: EXPANSÃO DA RECORRÊNCIA

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

Até onde vai?

PASSO A: EXPANSÃO DA RECORRÊNCIA

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

$$T(1) = 1$$

PASSO B: ENCONTRAR O PADRÃO

$$T(n) = T\left(\frac{n}{3}\right) + n$$

Padrão?

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

$$T(1) = 1$$

PASSO B: ENCONTRAR O PADRÃO

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

$$T(1) = 1$$

Padrão?

$$T\left(\frac{n}{3^i}\right) = T\left(\frac{n}{3^{i+1}}\right) + \frac{n}{3^i}$$

PASSO C: LIMITE DA EQUAÇÃO

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

$$T(1) = 1$$

Quantas expansões iremos fazer?

Padrão?

$$T\left(\frac{n}{3^i}\right) = T\left(\frac{n}{3^{i+1}}\right) + \frac{n}{3^i}$$

PASSO C: LIMITE DA EQUAÇÃO

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

$$T(1) = 1$$

Padrão?

$$T\left(\frac{n}{3^i}\right) = T\left(\frac{n}{3^{i+1}}\right) + \frac{n}{3^i}$$

$$\frac{n}{3^i} = 1$$

PASSO C: LIMITE DA EQUAÇÃO

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

$$T(1) = 1$$

Padrão?

$$T\left(\frac{n}{3^i}\right) = T\left(\frac{n}{3^{i+1}}\right) + \frac{n}{3^i}$$

$$\frac{n}{3^i} = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

PASSO D: REESCREVER EM FUNÇÃO DO NOVO N

$$T(n) = T\left(\frac{n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{\mathrm{n}}{9}\right) + \frac{n}{3}$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{\mathrm{n}}{27}\right) + \frac{n}{9}$$

$$T\left(\frac{n}{27}\right) = T\left(\frac{\mathrm{n}}{81}\right) + \frac{n}{27}$$

$$\vdots$$

$$T(1) = 1$$

Padrão?

$$T\left(\frac{n}{3^i}\right) = T\left(\frac{n}{3^{i+1}}\right) + \frac{n}{3^i}$$

$$\frac{n}{3^i} = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

PASSO D: REESCREVER EM FUNÇÃO DO NOVO N

$$T(3^i) = T(3^{i-1}) + 3^i$$

$$T(3^{i-1}) = T(3^{i-2}) + 3^{i-1}$$

$$T(3^{i-2}) = T(3^{i-3}) + 3^{i-2}$$

$$T(3^{i-3}) = T(3^{i-4}) + 3^{i-3}$$

$$\vdots$$

$$T(3^{i-i}) = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

PASSO D: REESCREVER EM FUNÇÃO DO NOVO N

$$T(3^i) = T(3^{i-1}) + 3^i$$

$$T(3^{i-1}) = T(3^{i-2}) + 3^{i-1}$$

$$T(3^{i-2}) = T(3^{i-3}) + 3^{i-2}$$

$$T(3^{i-3}) = T(3^{i-4}) + 3^{i-3}$$

$$\vdots$$

$$T(3^{i-(i-1)}) = T(3^{i-i}) + 3^{i-(i-1)}$$

$$T(3^{i-i}) = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

PASSO D: REESCREVER EM FUNÇÃO DO NOVO N

$$T(3^i) = T(3^{i-1}) + 3^i$$

$$T(3^{i-1}) = T(3^{i-2}) + 3^{i-1}$$

$$T(3^{i-2}) = T(3^{i-3}) + 3^{i-2}$$

$$T(3^{i-3}) = T(3^{i-4}) + 3^{i-3}$$

$$\vdots$$

$$T(3^{i-(i-1)}) = T(3^{i-i}) + 3^{i-(i-1)}$$

$$T(3^{i-i}) = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

$$T(3^i) = \sum_{j=0}^{i} 3^j$$

PASSO D: REESCREVER EM FUNÇÃO DO NOVO N

$$T(3^i) = T(3^{i-1}) + 3^i$$

$$T(3^{i-1}) = T(3^{i-2}) + 3^{i-1}$$

$$T(3^{i-2}) = T(3^{i-3}) + 3^{i-2}$$

$$T(3^{i-3}) = T(3^{i-4}) + 3^{i-3}$$

$$\vdots$$

$$T(3^{i-(i-1)}) = T(3^{i-i}) + 3^{i-(i-1)}$$

$$T(3^{i-i}) = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

$$T(3^i) = \sum_{j=0}^{i} 3^j$$

PASSO D: REESCREVER EM FUNÇÃO DO NOVO N

$$T(3^i) = T(3^{i-1}) + 3^i$$

$$T(3^{i-1}) = T(3^{i-2}) + 3^{i-1}$$

$$T(3^{i-2}) = T(3^{i-3}) + 3^{i-2}$$

$$T(3^{i-3}) = T(3^{i-4}) + 3^{i-3}$$

$$\vdots$$

$$T(3^{i-(i-1)}) = T(3^{i-i}) + 3^{i-(i-1)}$$

$$T(3^{i-i}) = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

$$T(3^i) = \sum_{j=0}^{i} 3^j$$

RELEMBRANDO ....

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\boxed{\sum_{i=0}^{k} a^i = \frac{a^{k+1}-1}{a-1} (a \neq 1)}$$

$$\sum_{i=0}^{k} 2^k = 2^{k+1} - 1$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=0}^{k} \frac{1}{2^i} = 2 - \frac{1}{2^k}$$

PASSO E: RESOLVER O SOMATÓRIO

$$T(3^i) = T(3^{i-1}) + 3^i$$

$$T(3^{i-1}) = T(3^{i-2}) + 3^{i-1}$$

$$T(3^{i-2}) = T(3^{i-3}) + 3^{i-2}$$

$$T(3^{i-3}) = T(3^{i-4}) + 3^{i-3}$$

$$\vdots$$

$$T(3^{i-(i-1)}) = T(3^{i-i}) + 3^{i-(i-1)}$$

$$T(3^{i-i}) = 1$$

$$n = 3^i$$

$$log_3(n) = i$$

$$T(3^i) = \sum_{j=0}^{i} 3^j$$

$$= \frac{3^{i+1} - 1}{3 - 1}$$

PASSO E: RESOLVER O SOMATÓRIO

$$T(3^i) = \sum_{j=0}^{i} 3^j$$

$$= \frac{3^{i+1} - 1}{3 - 1} = \frac{3^{log_3(n)+1} - 1}{2}$$

$$= \frac{3 \,.\, 3^{log_3(n)} - 1}{2} = \frac{3n - 1}{2}$$

EXERCÍCIOS

Resolva as seguintes equações de recorrência:

a) $$T(n) = \begin{cases} 1, & n = 1 \\ T\left(\frac{n}{2}\right) + 1, & n \geq 2 \end{cases}$$

b) $$T(n) = \begin{cases} 0, & n = 1 \\ 2T\left(\frac{n}{2}\right) + \mathrm{n}, & n \geq 2 \end{cases}$$